

El diseñar tests es un mecanismo efectivo para prevenir errores. El proceso de crear tests permite descubrir y eliminar problemas en todas las etapas de desarrollo.

Desarrollo tradicional vs Desarrollo Ágil (TDD)

1- Diseñar

2- Desarrollar

3- Probar



1- Diseñar

2- Probar

3- Desarrollar

Obligado a entender lo q' programan

En el desarrollo tradicional, las pruebas unitarias se realizan luego de escribir el código —> Asegurar calidad —> Enfoque Reactivo

En el desarrollo ágil, se escriben las pruebas unitarias antes q' el código -> Asegurar y controlar calidad —> Enfoque Proactivo

TDD

Involucra dos prácticas: escribir pruebas primero y refactorización

Pasos:

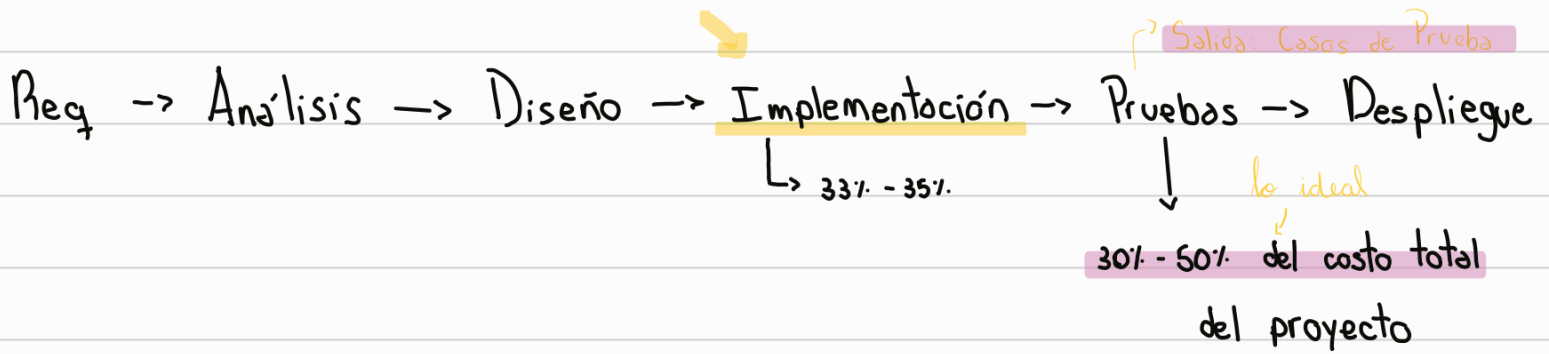
- 1) Escribir un test
- 2) Hacer q' compile
- 3) Hacer q' el test falle (red)
- 4) Hacer cambios para q' el test pase
- 5) Correr el test para que pase (green)
- 6) Refactor

Leyes TDD:

- No escribir código hasta escribir un test que falle
- No escribir más tests de los necesarios
- No escribir + código en producción para q' el test pase



TDD -> técnica de desarrollo —> desarrollo guiado por pruebas



Costo + significativo —> esfuerzo $\cong$ personas

identificación
Caso de Prueba : su propósito es determinar si se cumplen los CA.
(*Confirmación de la US*) requieren Datos, pre condiciones (datos de antemano -> req.)
Surgen de los req. ¿Por qué no avanzar (No es - esfuerzo, es - tiempo) el trabajo ? de calendario

Agilismo -> adelantar el pensamiento de testing

ATDD : Aceptación

Niveles de pruebas +
UAT -> Sistema o versión -> Integración -> Unitarias

Refactoring : mejorar la calidad de los componentes

